ZENITH SCHOOL OF AI / OPEN-SOURCE EVALUATION

FOOD-DELIVERY OPERATIONS

OrderOps

Zomato-Inspired Order Failure, Cancellation & Refund Reconciliation

FORMAT	DURATION	LANGUAGE	DATA
Individual	24-hour individual evaluation	Python 3.10+	Synthetic / offline

Official evaluation repository: https://github.com/sunitazenith07-lang/OrderOps-Open-Source-Evaluation

Open the repository in a browser, then read `STUDENT_PROBLEM_STATEMENT.md` and the README before changing code.

1. Scenario

In a food-delivery workflow of the kind customers may associate with Zomato, payment can be confirmed before a restaurant accepts an order. The restaurant may reject it; customers may cancel before dispatch; and refund notifications can arrive late or more than once. The service must not duplicate a refund or confuse a partial refund with a completed refund.

This is an independent, simulated, offline exercise using synthetic data. Zomato is referenced only as a familiar industry example; this is not Zomato software, an official Zomato assessment, or an affiliated project. It does not use real customers or payment APIs.

2. Required behaviour

1. Process the supplied events: `payment_confirmed`, `restaurant_accepted`, `restaurant_rejected`, `cancel_requested`, `dispatch_started`, `order_delivered` and `refund_processed`.
2. Enforce valid transitions. A cancellation after dispatch must not silently change the order to CANCELLED.
3. Use `event_id` for idempotency. Replaying an event must not change order or refund state a second time.
4. Use integer paise. Refund amounts must be positive, cannot exceed the outstanding refundable amount, and apply only to eligible cancelled/rejected orders that were paid.
5. Track `refund_due_paise` and `refunded_paise` separately. Partial refunds remain partial; set payment status to REFUNDED only when the full due amount has been processed.
6. Reconcile internal and provider summaries by order ID. Report missing records, amount mismatches and status mismatches separately and deterministically.
7. Handle empty input, missing fields, unknown IDs, invalid event types and duplicates without corrupting state.

3. Setup and first checks

Use Python 3.10 or newer and Git. From the repository root, run the following commands in order:

```
python -m pip install -r requirements-dev.txt
python -m pytest -q
python -m main
```

4. Work to submit

- Read the source and run the starter tests before editing.
- Treat the requirements above as the acceptance contract. Investigate failing tests, trace root causes, and correct the underlying behaviour; do not hard-code fixture output.
- Add regression tests for fixes and document at least five edge cases in `TESTING.md`.
- Submit one pull request against the original repository with code, tests, a root-cause summary and known limitations. Do not merge it.

- Include `ORDER_FLOW_NOTES.md` with one example event sequence, expected states and three edge cases.

5. Optional stretch work

Choose at most one stretch item, and only after the required behaviour is correct. Stretch work is not required for a strong score.

- Export an awaiting-refund CSV report.
- Flag out-of-order events.
- Summarise orders by state.

6. Testing and GitHub Actions

Some visible tests are expected to fail on the untouched starter because key behaviours are incomplete. The exact initial test count is not fixed. GitHub Actions runs on pushes and pull-request updates and reports visible test counts. A red check before your fixes is expected; make the visible tests pass without deleting, skipping, weakening or bypassing them. Do not alter the workflow to evade failures.

7. Individual-work policy

AI chatbots, AI coding assistants, generated patches/tests and asking another person to write or debug your solution are prohibited. Ordinary non-AI language/framework documentation is allowed. Do not upload challenge code or data to external services. You must be able to explain your changes and make a small live modification if asked. A code walkthrough can support integrity checks, but no automated detector can prove whether AI was or was not used.

8. Evaluation rubric - 100 marks

Criterion	Marks
Codebase understanding and state-flow explanation	**15**
Root-cause analysis and correctness	**25**
State transitions, idempotency and refund safety	**25**
Regression testing and edge cases	**15**
Code quality and maintainability	**10**
Pull request, notes and live explanation	**10**
Total	**100**

Technical expectation: Marks reward correct reasoning and behaviour that matches this written contract, not line count. A green test suite alone does not guarantee full marks; be ready to explain your choices, test strategy and known limitations.

The objective is to preserve valid order/payment state and avoid duplicate or ineligible refunds.

This is a simulated startup-style challenge inspired by common product/operations problems. It is not a confidential or official problem statement from any named company. All datasets are artificial.